

Ain Shams University

Faculty of Computer and Information Sciences

Resala Charity Management System

Phase 1 Report

Course: CSE336 — Software Design Patterns

Instructor: Dr. Raghda Ali

Student	ID
Sharqawy Wael	23P0360
Zyad El-Nemr	23P0013
Islam Nashaat	23P0077
Mohammed Osama	23P0374
Youssef Tarek	23P0177
Fady Joseph	23P0124

4 November 2026

Contents

1 Project overview **1**
1.1 Implemented modules 1
1.2 Objectives 1

2 Design and architecture **1**
2.1 Main responsibilities 2
2.2 One operation across all layers 2

3 Class diagram **2**

4 Phase 1 design patterns **3**
4.1 Strategy pattern 3
4.2 Observer pattern 3
4.3 Decorator pattern 4
4.4 Factory pattern 4
4.5 Singleton pattern 4
4.6 MVC 4

5 Client requirements and lookup CRUD **5**
5.1 Client requirements 5
5.2 Lookup-table CRUD: event types 5

6 Conclusion **6**

1 Project overview

Resala Charity Management System is a desktop application for organizing the daily work of a non-profit organization. It brings donations, donors, volunteers, events, beneficiaries, communication, and reference data into one local system. The project has a graphical interface and a console interface, uses a local database, and follows an MVC structure.

The goal of Phase 1 is to establish a clear structure and to apply the required design patterns. The interface, control logic, business rules, and data storage are separated, so the project can be extended in later phases without rewriting the existing modules.

Phase 1 scope

The current version supports the main management screens, local storage, reference-data maintenance, MVC organisation, and the required patterns: Strategy, Observer, Decorator, Factory, and Singleton.

1.1 Implemented modules

- **Donations:** donors and donations of three types (online, check, in-kind), receipts, filtering, and a yearly tax statement. Payment is simulated for demonstration.
- **Volunteers:** skills, availability, worked hours, assignment to suitable events, and certificates.
- **Events:** creation, editable event types, attendee registration, attendance, and automatic notifications when an event changes.
- **Beneficiaries:** needs, aid distribution, distribution history, and a selectable distribution policy.
- **Communication:** email, SMS, and social channels, selectable at runtime and simulated in this version.
- **Reference data:** event types and skills, with full create, read, update, and delete operations.

1.2 Objectives

The brief asks for flexibility, maintainability, and reusability. The project meets these goals in three ways:

- **Flexibility:** new donation types, notification channels, and distribution policies can be added without changing the services that use them.
- **Maintainability:** each layer has one responsibility, and screens never contain business rules or storage code.
- **Reusability:** receipts, notifications, and storage interfaces are written once and composed where they are needed.

2 Design and architecture

The system follows an MVC structure. Screens display data and collect user actions. Controllers receive these actions and pass them to services. Services apply the business rules and use repository interfaces to read and write data. The repository implementations contain the storage details.

The same structure supports both interfaces. The graphical interface and the console interface use the same controllers and services, so a business rule is written once and behaves identically in both. This is the practical proof of the MVC boundary.

Placeholder: layered architecture diagram

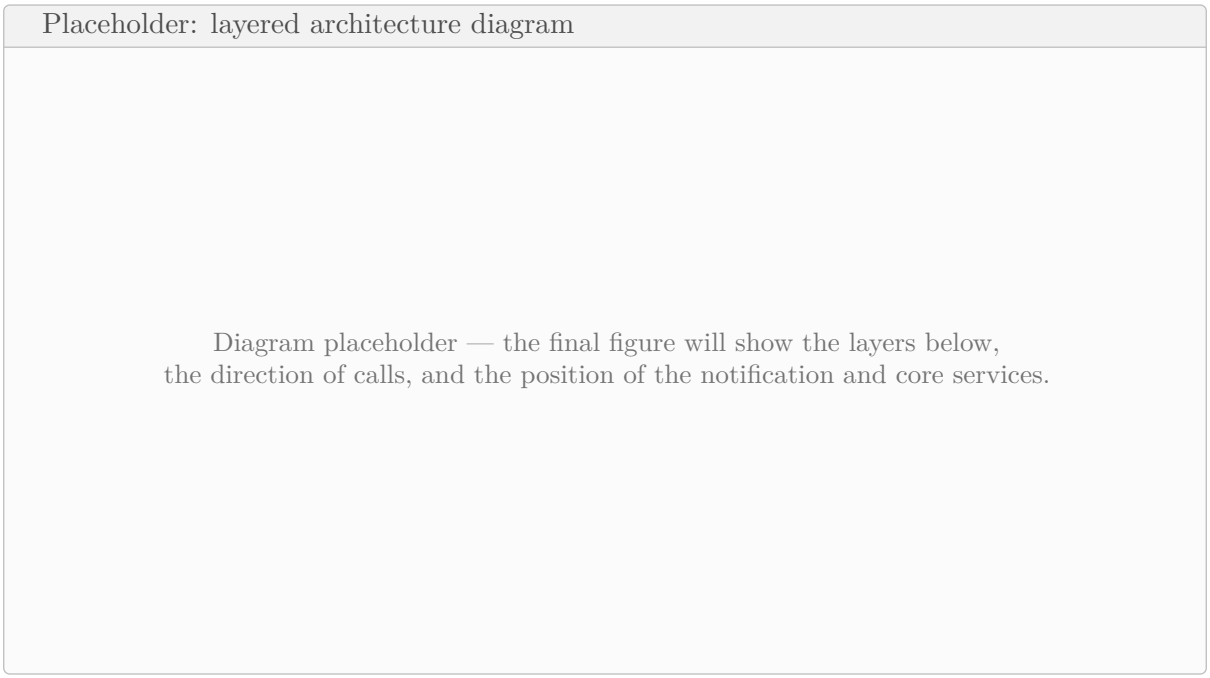


Figure 1: Layered architecture of the system.

2.1 Main responsibilities

Layer	Responsibility
Views	Display data, collect input, and show feedback to the user.
Controllers	Receive requests from the screens and delegate them to the services.
Services	Validate data and enforce the rules of each module.
Repositories	Define the storage operations without exposing storage details.
Storage implementations	Carry out the repository operations on the local database.
Core services	Provide shared configuration, database access, transactions, and audit logging.

Table 1: Responsibility of each layer.

2.2 One operation across all layers

When a donation is recorded, the screen collects the form values, the controller passes the request to the donation service, and the service applies the rules and asks the factory to create the correct donation. The repository saves the result, the service notifies its observers, and the screen refreshes. The console interface performs the same operation through the same controller and service, so no rule is duplicated between the two interfaces.

3 Class diagram

The class diagram covers the whole project. Following the course guidance, it shows the main classes with only their important fields and methods, not every accessor. The diagram is organised by module, and the six Phase 1 patterns are marked with a colour legend so that each pattern can be located immediately.

Placeholder: full-project class diagram

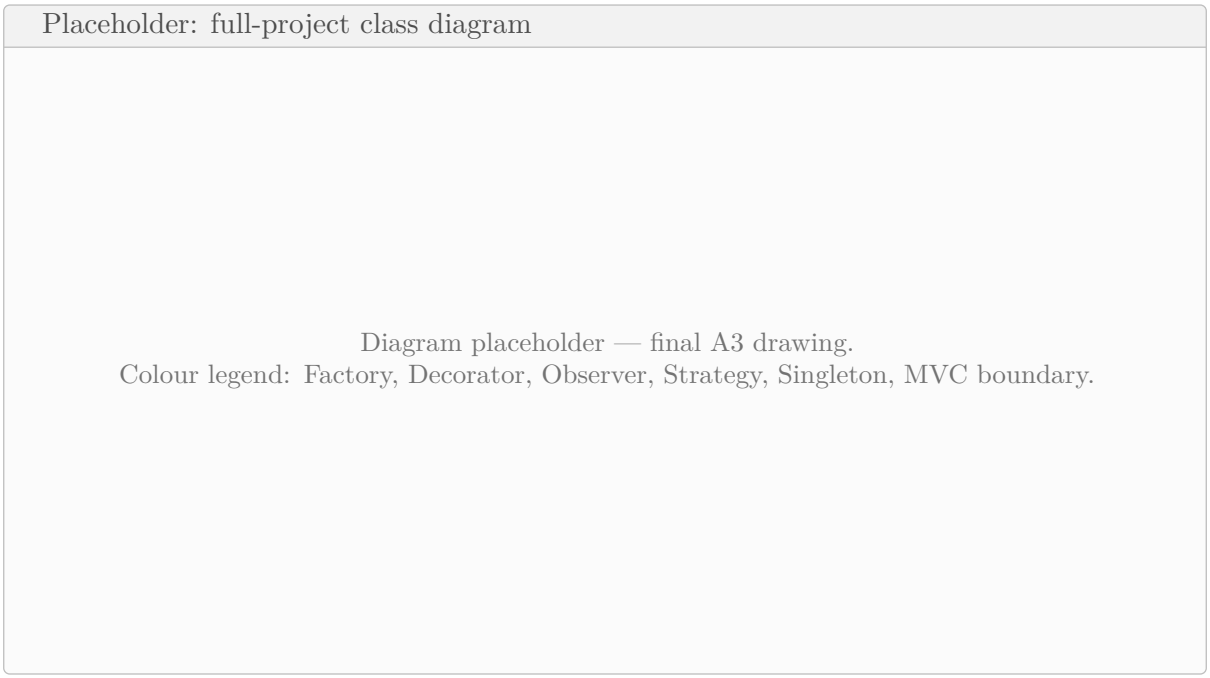


Figure 2: Complete class diagram of the Resala Charity Management System.

The diagram is divided into eight areas: the entry and interface, the controllers, donations and receipts, events and volunteers, beneficiaries, notifications, persistence, and core services. It shows the relationships that carry the design, not only the data model: the factory with its product family, the decorator chain around the basic receipt, the subjects with their observers, the notifier with its interchangeable strategies, the shared services marked as singletons, and the dashed MVC boundary between the four layers.

4 Phase 1 design patterns

This section describes each required pattern, how it is applied in the project, and why it was chosen. The same structure is used for all six patterns so that the design can be compared and defended consistently.

4.1 Strategy pattern

Intent. Define a family of algorithms, encapsulate each one, and make them interchangeable at runtime.

How it is applied. The communication module defines one notification strategy with three implementations: email, SMS, and social media. The notifier holds the current strategy and forwards every message to it. When the user changes the channel in the Messages screen, the notifier replaces the strategy, and every later message goes through the new channel. The same idea is used a second time in the beneficiary module, where the distribution policy can be changed between equal, by-need, and first-come-first-served allocation.

Why this pattern. Adding a new channel or a new distribution policy requires one new class that follows the same interface. The notifier, the controllers, and the screens do not change. This is the clearest demonstration of the flexibility objective in the brief.

4.2 Observer pattern

Intent. Define a one-to-many dependency so that when one object changes state, all its dependents are notified automatically.

How it is applied. The donation service and the event service act as subjects. After a donation is recorded, the service notifies its observers: one adds the entry to the activity feed, and another sends a thank-you message through the notifier. When an event is updated, opened, closed, or reminded, the event service notifies its observers, and the event observer informs the registered attendees.

Why this pattern. The service that records a donation has no reference to the feed, the notifier, or any screen. A new reaction, such as a loyalty tracker, is added as a new observer without editing the service. This keeps the module closed for modification and open for extension.

4.3 Decorator pattern

Intent. Attach additional responsibilities to an object dynamically, without creating a subclass for every combination.

How it is applied. A receipt is built from a base receipt and optional decorators. The tax decorator adds the tax section, and the thank-you decorator adds a thank-you message. A monetary donation can therefore receive tax information while every receipt can include a thank-you message.

Why this pattern. Two optional features would otherwise require four subclasses: basic, basic with tax, basic with thank-you, and both. With the decorator, the client composes the sections it needs at runtime, and a third optional section would add one class rather than doubling the hierarchy again.

4.4 Factory pattern

Intent. Provide one creation point so that the caller does not depend on the concrete product classes.

How it is applied. The donation factory receives the donation type and creates the matching donation: online, check, or in-kind. Each type has its own validation; an online donation needs a positive amount, a check needs a check number, and an in-kind donation needs an item and a quantity. The factory keeps these rules in one place and returns the abstract donation to the service. A second, smaller use of the pattern creates the family of console screens, so the interface can be assembled without naming concrete screens.

Why this pattern. Adding a new donation type requires one new class and one branch in the factory. The service, the controllers, and the screens stay unchanged, and the creation rules do not leak into the interface code.

4.5 Singleton pattern

Intent. Ensure that a class has exactly one instance and provide a global access point to it.

How it is applied. Three shared services are singletons: the database manager, the configuration manager, and the audit log. The database manager owns the single connection and creates the schema once. The configuration manager loads the application settings once. The audit log collects events from every module in one place.

Why this pattern. Several connections to the same local database would cause locking problems, and several audit logs would lose events. A private constructor and a single access point guarantee one instance without passing it through every constructor.

4.6 MVC

Intent. Separate presentation, control, and domain logic so that each can change independently.

How it is applied. MVC is used across the main modules. A screen handles the interface, a controller receives the screen request, and a service performs the operation. The repository sits below the service and hides the storage. Table 2 summarises the roles and what each one must not do.

Role	Example	Must not do
Model	Donation, Volunteer, Event	Contain storage code or interface code.
View	Donation screen, console donation menu	Contain business rules or open a database connection.
Controller	Donation controller	Print to the console or read keyboard input directly.
Service	Donation service	Know which screen called it.
Repository	Donation repository	Expose storage details to the service.

Table 2: The MVC roles and their boundaries.

Why this pattern. The strongest evidence is that the project has two complete user interfaces, graphical and console, built on the same controllers and services. If the business rules had been placed in the screens, the second interface would have required duplicating them.

5 Client requirements and lookup CRUD

5.1 Client requirements

The requirements were written from the point of view of the staff who operate the system, not from the point of view of the code. Each one describes a problem in the current manual process and the outcome the client needs. The last column links the need to the module that delivers it.

ID	Client need and expected outcome	Addressed by
CR1	**Year-end statements.** At year end, staff rebuild each donor's total by hand. The client needs an accurate statement for any donor and year, covering monetary donations only.	**FR-D6**; Donations → Yearly Statement
CR2	**Safe volunteer assignment.** Coordinators assign volunteers from memory, which causes skill mismatches and no-shows. The client needs every assignment checked against availability and the required skill.	**FR-V4**; Volunteers → Register to Event
CR3	**Volunteer certificates.** Certificates are calculated by hand and issued late. The client needs a certificate produced only after 20 verified hours, showing the correct total.	**FR-V5, FR-V6**; Volunteers → Print Certificate
CR4	**Event changes.** Attendees are not told when an event changes, so some travel for nothing. The client needs every registered attendee informed automatically.	**FR-E6**; automatic, and Events → Send Reminder
CR5	**Aid history.** Aid is recorded on paper, so the branch cannot say who received what, when, or whether it was already given. The client needs one history per beneficiary.	**FR-B3, FR-B4**; Beneficiaries → Distribute Aid / History
CR6	**Staff-maintained lists.** Every new event type or skill currently needs a developer. The client needs staff to add, rename and remove these values themselves without breaking existing records.	**FR-R1 to FR-R5**; Lookups
CR7	**One message screen.** Supporters use different channels, and the branch cannot run a separate tool for each. The client needs one message screen where the channel can be changed immediately.	**FR-C1 to FR-C3**; Messages

Table 3: Client needs and their implementation.

5.2 Lookup-table CRUD: event types

Event type was selected as the Phase 1 lookup-table example. It is an editable value used when creating an event, and the same screen also supports skills. Following the lecturer's guidance, the lookup is treated as application data, and the four operations are mapped to the functions that use them.

Function or use case	EventType	Event	Interface entry point
Create a new event type	C	–	Lookups → Add
Browse all event types	R	–	Lookups table; event form
Rename an event type	U	–	Lookups → Rename
Delete an unused event type	D	R (guard)	Lookups → Delete
Assign a type when creating or editing an event	R	C / U	Events → Create / Edit

Table 4: CRUD matrix for the chosen lookup module.

The service enforces the following rules, so they apply to both interfaces:

- the name is required, trimmed, and must be unique;
- rename updates the stored value, so every event that references the type shows the new name;
- delete is refused when the type is already used by an event, which protects existing records;
- every operation refreshes the open screens, so the event form immediately shows the new list.

Skills use the same screen and the same four operations. A skill is referenced by the volunteer–skill relation, so the delete guard protects volunteer records in the same way. Two working tables show that the lookup screen is a general mechanism rather than a single hard-coded form.

6 Conclusion

Phase 1 provides a working base for the Resala Charity Management System. The project has a graphical interface, local storage, MVC separation, editable reference data, and the required Phase 1 patterns. The modules cover the core charity operations: donations, volunteers, events, beneficiaries, and communication.

The design leaves room for the next phase, where the remaining patterns and the admin pages can be added without rewriting the existing modules. The current limits are stated clearly: payment and messaging are simulated, authentication is not part of this phase, and need types are supported by the beneficiary module but not yet exposed on the lookup screen.